



Visualización de Información y Analítica Visual

— Hernán Valdivieso López (hfvdivieso@uc.cl) —

Clase 8: Tópicos Avanzados

Contenidos

- Visualización Analítica en la Era de la IA
 - Uso de agentes en el proceso de diseño.
 - Inteligencia Artificial Explicable (XAI).
- Streamlit - Desarrollo de *dashboard* con Python
 - ¿Qué es y qué cosas se pueden hacer?
 - ¿Cómo publicar esta herramienta?
 - Presentación de una herramienta
 - ¿Existen otras alternativas?

Visualización Analítica en la Era de la IA

¿Qué cambia para un analista
cuando aparecen los LLM y los
agentes?

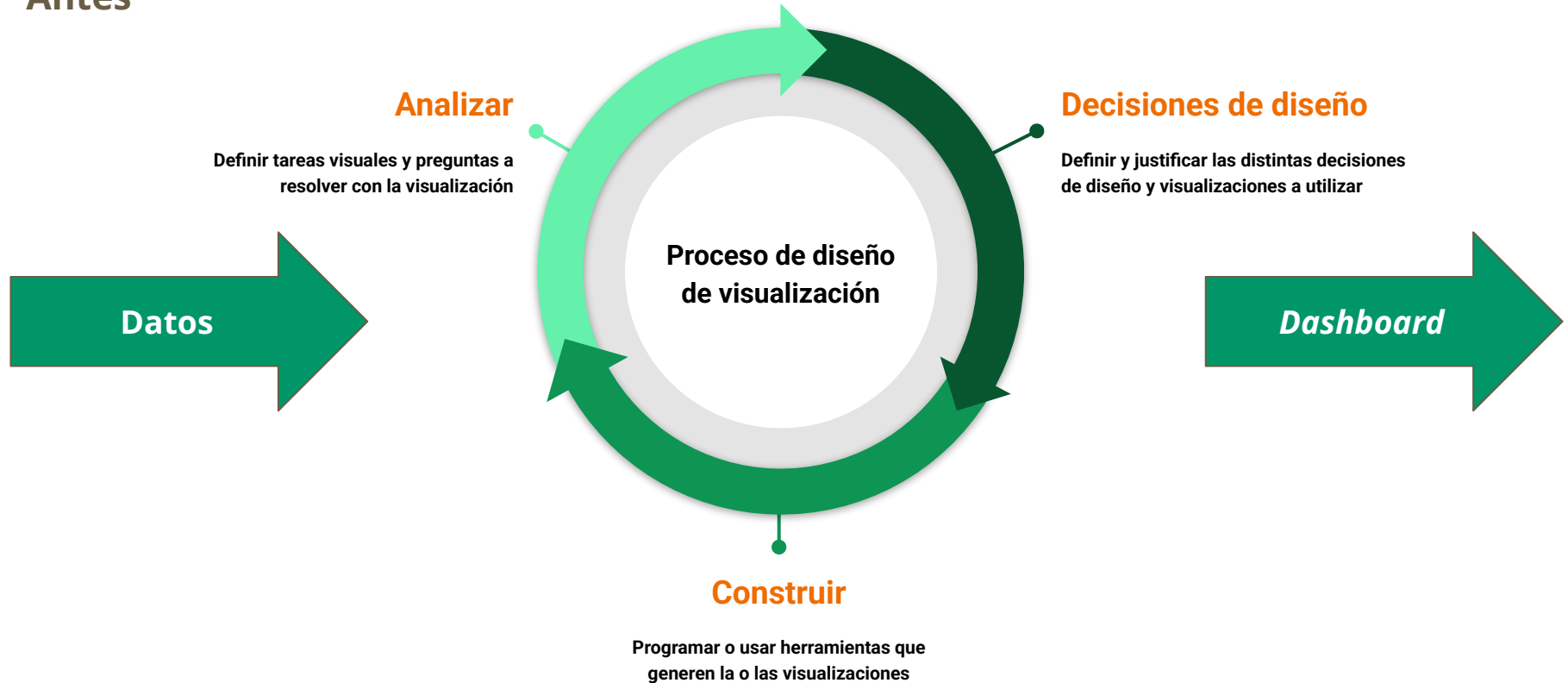
Visualización Analítica en la Era de la IA

Antes



Visualización Analítica en la Era de la IA

Antes



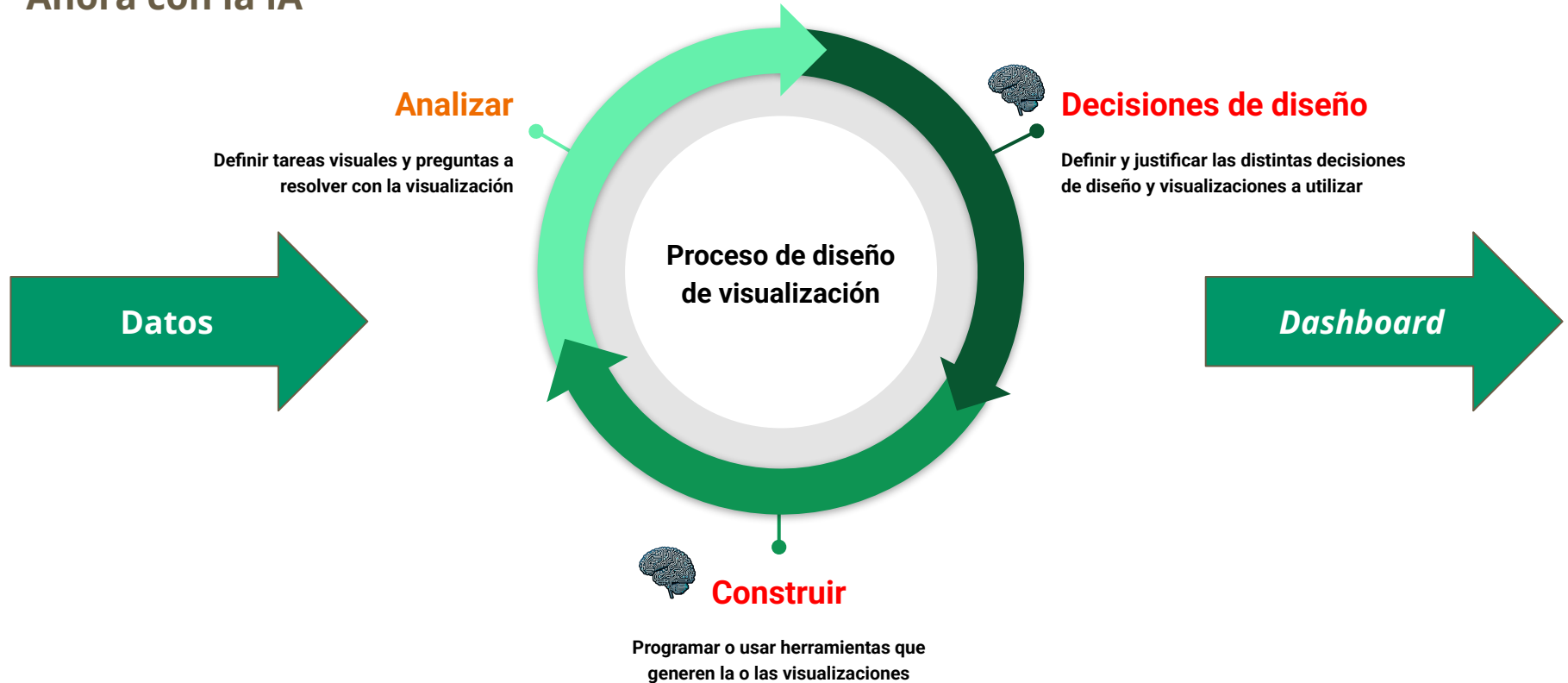
Visualización Analítica en la Era de la IA

Antes



Visualización Analítica en la Era de la IA

Ahora con la IA



Visualización Analítica en la Era de la IA

Ahora con la IA



Visualización Analítica en la Era de la IA

IA para escribir código → La IA actúa como asistente de programación.

- "Genera un gráfico de barras en Altair."
- "Convierte este gráfico de Matplotlib a Plotly."
- "Agrega etiquetas al gráfico"
- "Cambia la paleta de colores"

Visualización Analítica en la Era de la IA

😊😊 IA para escribir código → La IA actúa como asistente de programación. 😊😊

- Acelera el desarrollo.
- Favorece el aprendizaje.
- Favorece la experimentación.
- Permite enfocarse en el análisis.

Visualización Analítica en la Era de la IA

⚠️⚠️ IA para escribir código → La IA actúa como asistente de programación. ⚠️⚠️

- El código puede ser incorrecto o utilizar elementos deprecados.
- Puede elegir un tipo de gráfico poco apropiado para los datos.
- Puede realizar código redundante o poco eficiente.
- Recurrir a configuraciones que funcionan, pero no siguen buenas prácticas.
- Puede generar dependencia excesiva.

Visualización Analítica en la Era de la IA

⚠️⚠️ IA para escribir código → La IA actúa como asistente de programación. ⚠️⚠️

- El código puede ser incorrecto o utilizar elementos deprecados.
- Puede elegir un tipo de gráfico poco apropiado para los datos.
- Puede realizar código redundante o poco eficiente.
- Recurrir a configuraciones que funcionan, pero no siguen buenas prácticas.
- Puede generar dependencia excesiva.
- **La responsabilidad sigue siendo del analista**
 - Verificar que la visualización represente correctamente los datos.
 - Revisar escalas, colores, etiquetas y posibles interpretaciones erróneas.

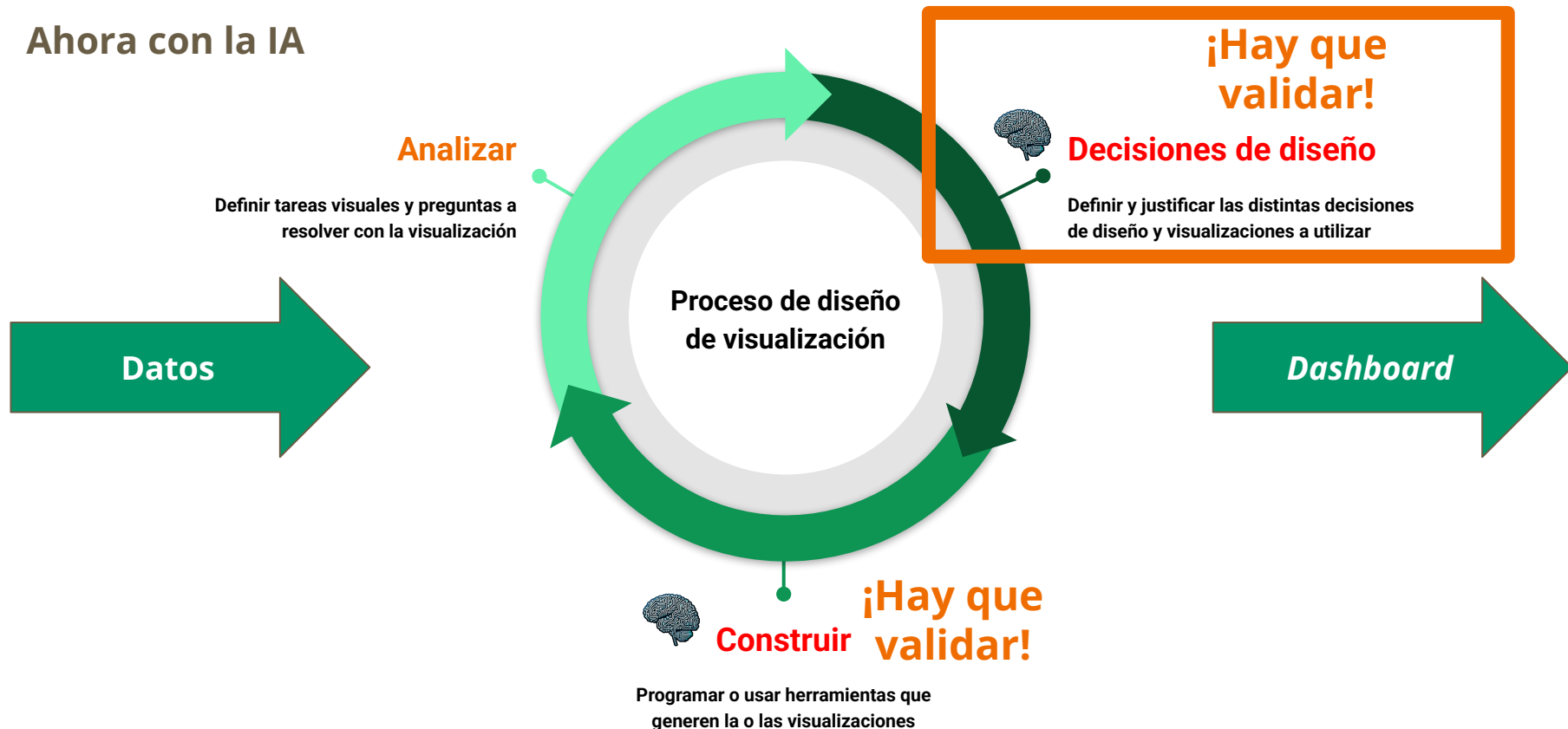
Visualización Analítica en la Era de la IA

Ahora con la IA



Visualización Analítica en la Era de la IA

Ahora con la IA



Visualización Analítica en la Era de la IA

IA para diseñar visualizaciones → La IA actúa como diseñador

- "¿Qué gráfico es el más adecuado para este *dataset*?"
- "Diseña un *dashboard* ejecutivo."
- "Propón una visualización para detectar valores atípicos."
- "Rediseña esta visualización para mejorar su legibilidad."

Visualización Analítica en la Era de la IA

😊😊 IA para diseñar visualizaciones → La IA actúa como diseñador 😊😊

- Genera propuestas de diseño rápidamente.
- Sugiere visualizaciones que quizás no habíamos considerado.
- Facilita la creación de múltiples alternativas para comparar.
- Ayuda a estructurar *dashboards* según distintos perfiles de usuarios.
- Reduce el tiempo dedicado al diseño inicial.
- Permite iterar el diseño mediante conversación.

Visualización Analítica en la Era de la IA

⚠ ⚠ IA para diseñar visualizaciones → La IA actúa como diseñador ⚠ ⚠

- Puede escoger un gráfico atractivo, pero poco adecuado para la tarea.
- No conoce automáticamente el contexto del negocio.
- Tiende a sobrecargar los *dashboards* con demasiada información.
- Puede ignorar principios de percepción visual y buenas prácticas.
- No reemplaza el juicio del analista ni la validación con usuarios.

Visualización Analítica en la Era de la IA

IA para diseñar visualizaciones → 💡 Consejos 💡

Pedir tareas analíticas, no gráficos

Prompt débil

Haz un *dashboard* de ventas o haz un gráfico de barra.

Prompt efectivo

Diseña una visualización que permita detectar anomalías, comparar regiones e identificar tendencias temporales.

Visualización Analítica en la Era de la IA

IA para diseñar visualizaciones → 💡 Consejos 💡

Establecer restricciones de diseño

- Prioriza posición sobre color.
- Evita gráficos 3D.
- No uses gráficos de torta con más de cinco categorías.
- Usa el color solo para destacar excepciones.
- Máximo cinco visualizaciones simultáneas.

Visualización Analítica en la Era de la IA

IA para diseñar visualizaciones → 💡 Consejos 💡

Obligar a justificar el diseño

Pide que la IA explique su razonamiento antes de construir el *dashboard*. Solicita que indique:

- Usuario objetivo
- Tareas principales
- Justificación de las decisiones de diseño

Obliga al agente a razonar antes de diseñar y ustedes pueden validar si esa justificación se alinea con sus expectativas iniciales.

Visualización Analítica en la Era de la IA

IA para diseñar visualizaciones → 💡 Consejos 💡

Diseñar *dashboards* para responder preguntas

Incluir preguntas que te interesa responder

- ¿Qué pasó en las ventas en X fecha?
- ¿Qué modelo es mejor?
- ¿hay una correlación entre algunos atributos?
- ¿Qué acción tomar dado ... ?

Visualización Analítica en la Era de la IA

IA para diseñar visualizaciones → 💡 Consejos 💡

Pedir una autoevaluación del resultado

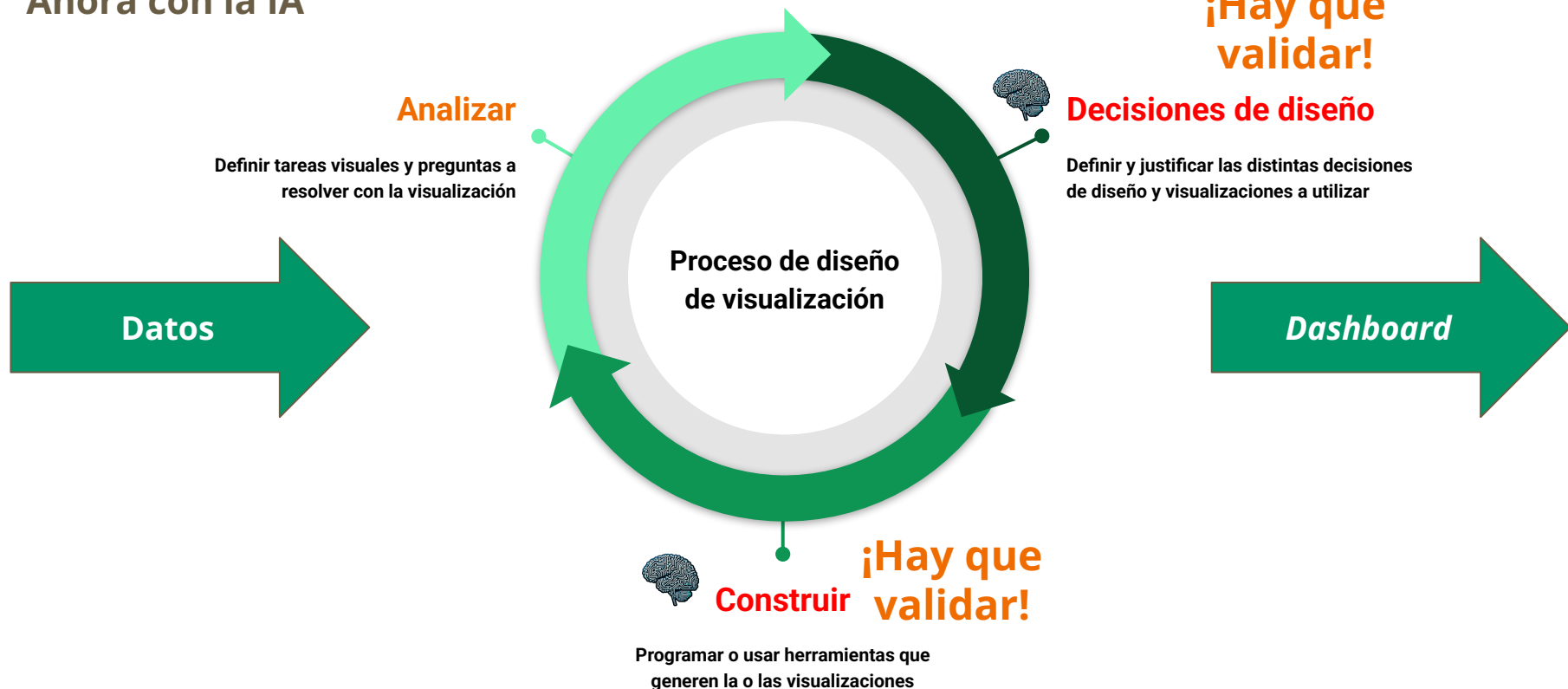
Antes de aceptar el dashboard, solicita que la IA lo critique. Por ejemplo:

- Adecuación tarea-visualización
- Carga cognitiva
- Consistencia de escalas
- Uso del color
- Tiempo estimado hasta encontrar el *insight* principal

Importante: la IA nunca estará conforme con el resultado. Uno debe saber cuando parar.

Visualización Analítica en la Era de la IA

Ahora con la IA



Visualización Analítica en la Era de la IA

Ahora con la IA



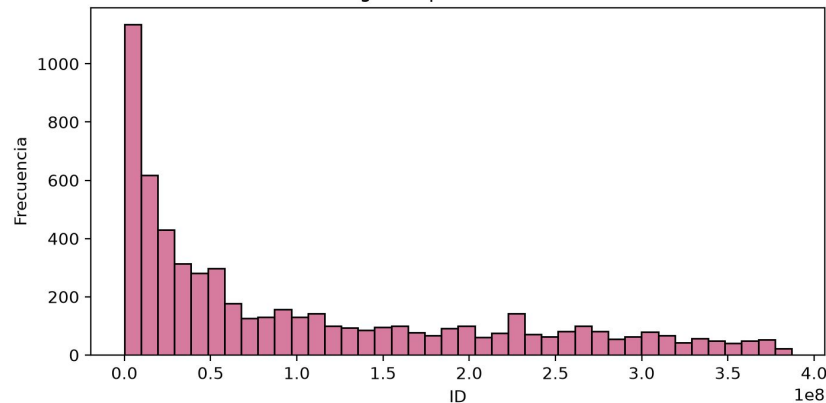
Visualización Analítica en la Era de la IA

Ejemplos programados por la IA que exigen iteración

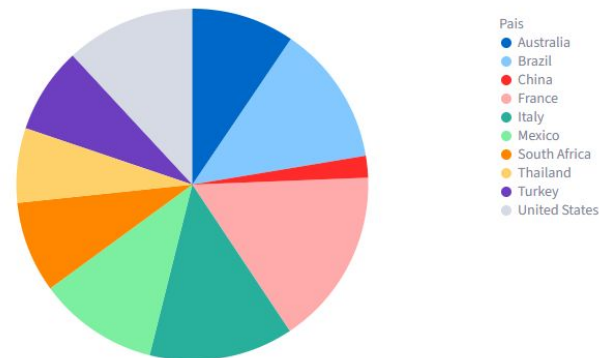
Puntaje promedio de comunicación vs localización por país



Histograma por ID de anfitrión



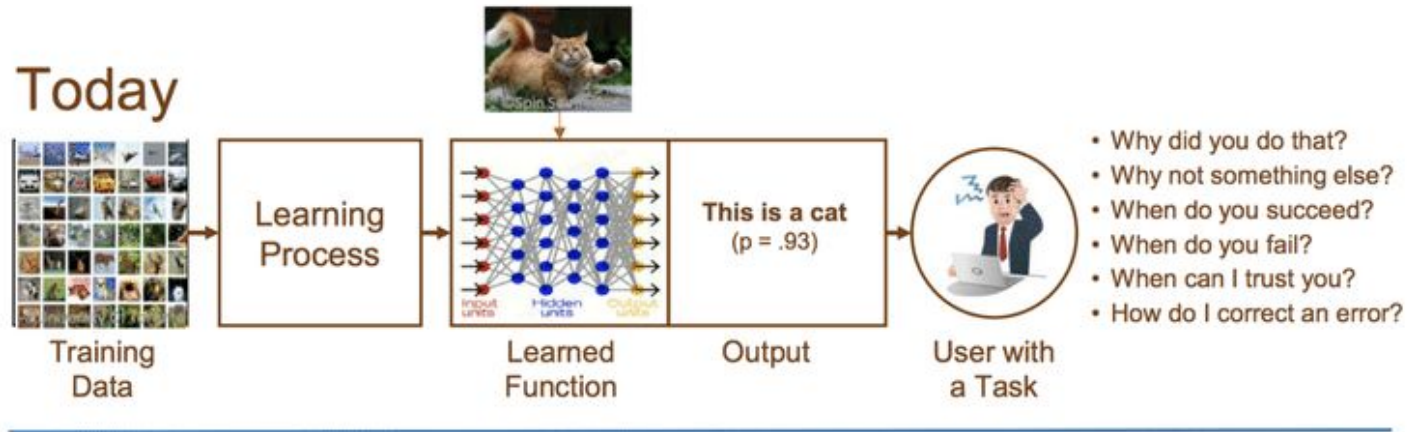
Proporción Airbnb por país



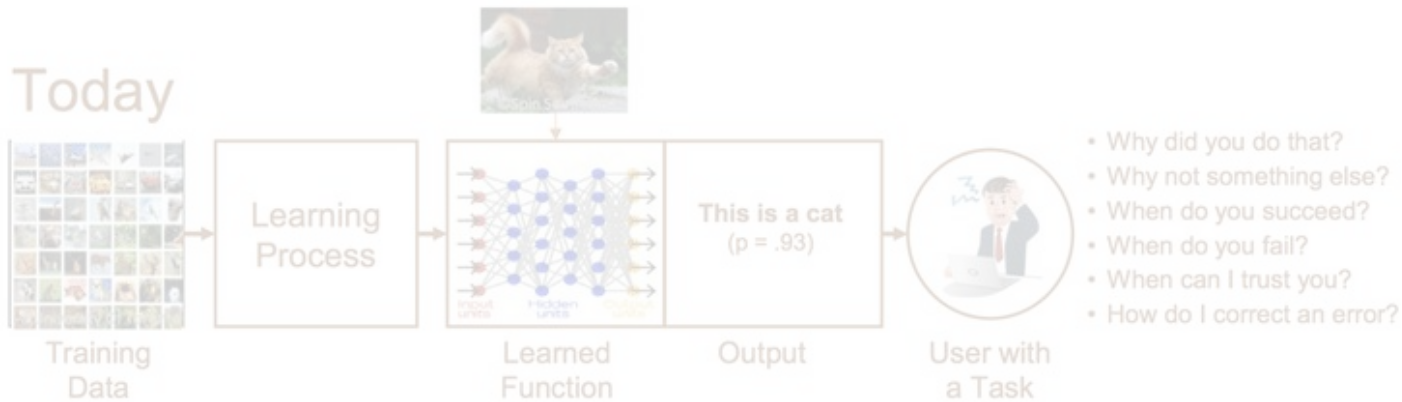
Visualización Analítica en la Era de la IA

Explainable Artificial Intelligence
(XAI)

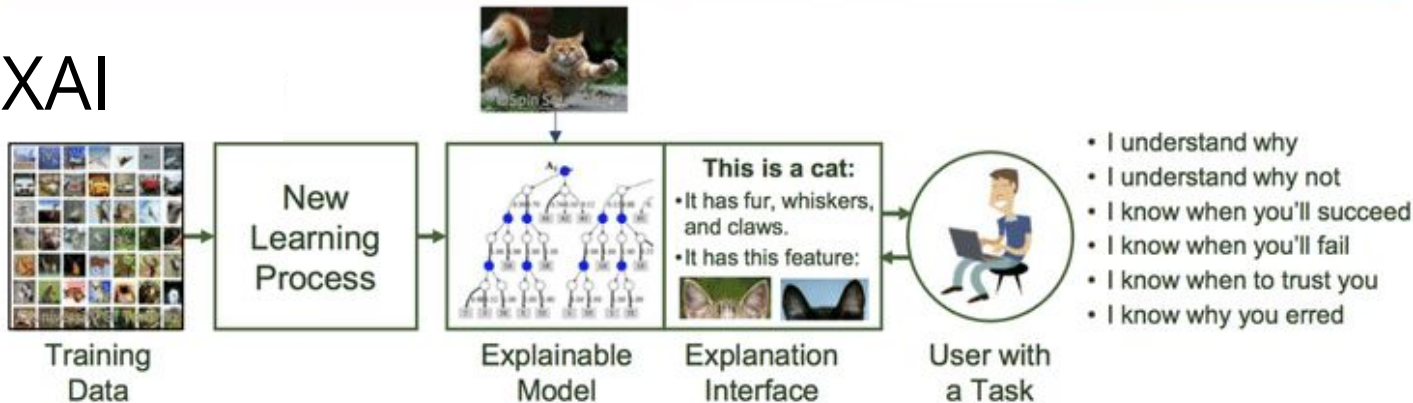
Explainable Artificial Intelligence (XAI)



Explainable Artificial Intelligence (XAI)



XAI



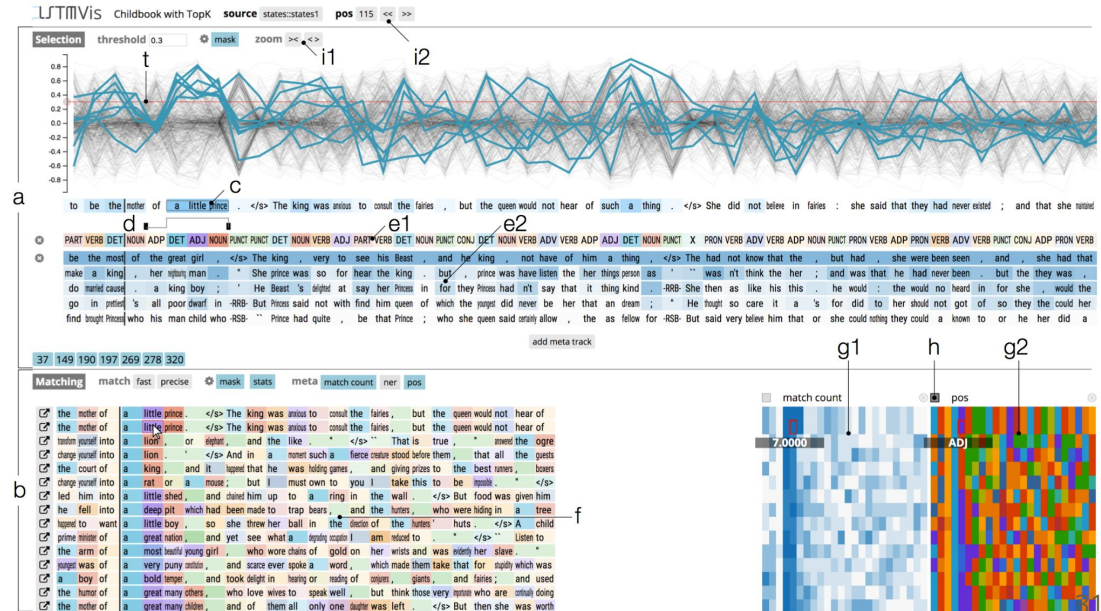
Explainable Artificial Intelligence (XAI)

- Término acuñado por [David Gunning el 2016](#).
- Surge de la necesidad de entender un poco más estas "cajas negras".
- Es posible diferenciar 2 tipos de explicaciones: globales y locales.

Explainable Artificial Intelligence (XAI)

- Término acuñado por [David Gunning el 2016](#).
- Surge de la necesidad de entender un poco más estas "cajas negras".
- Es posible diferenciar 2 tipos de explicaciones: [globales](#) y locales.

Global: permiten comprender el **razonamiento general** del sistema o modelo de IA que conduce a los diferentes resultados posibles



Explainable Artificial Intelligence (XAI)

- Término acuñado por [David Gunning el 2016](#).
- Surge de la necesidad de entender un poco más estas "cajas negras".
- Es posible diferenciar 2 tipos de explicaciones: globales y [locales](#).

Local: permiten comprender las razones de una **respuesta específica** (clasificación, recomendación, entre otros) que entregó el sistema o modelo de IA

Probabilidad para ser clasificada cómo noticia económica: 85%

Diputados de oposición proponen bono para compensar histórica caída del fondo E

Conocido como el menos riesgoso, las y los pensionados se sorprendieron con el 10,4% de caída acumulada que presenta este fondo. Mientras parlamentarios de oposición entregaron una propuesta de reparación, expertos del mercado aseguran que el origen de la pérdida se encuentra en iniciativas aprobadas por el propio Congreso.

Explainable Artificial Intelligence (XAI) - Exp. Locales

- Lo más utilizado son explicaciones locales, puesto que son más "sencillas" de entender.
- Es posible identificar 3 principales estrategias de explicación local:
 - **Feature importance:** una de las estrategias más utilizadas e indica cuáles datos de entrada son los más importantes para el sistema o modelo de IA, ya que tienen un impacto, positivo o negativo, en el resultado.
 - **Analogías:** esta estrategia resalta instancias similares o muy diferentes a los datos de entrenamiento para que sirvan de apoyo o contraejemplo en la explicación.
 - **Contrafactuales:** esta estrategia busca un punto cercano a los datos de entrada (por ejemplo, algún valor umbral) para los cuales la respuesta del sistema o modelo es alterada.

Explainable Artificial Intelligence (XAI) - Exp. Lcales

- Lo más utilizado son explicaciones locales, puesto que son más "sencillas" de entender.
- Es posible identificar 3 principales estrategias de explicación local:
 - **Feature importance**



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



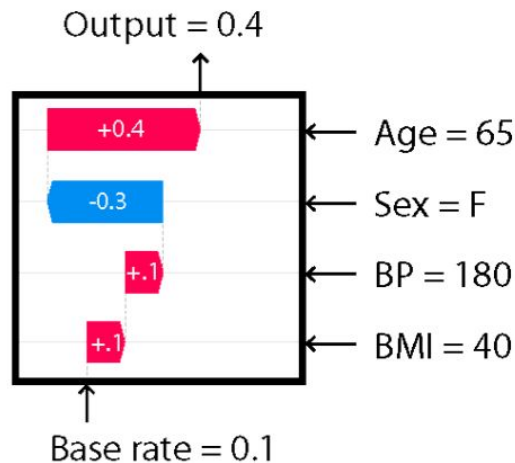
A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.

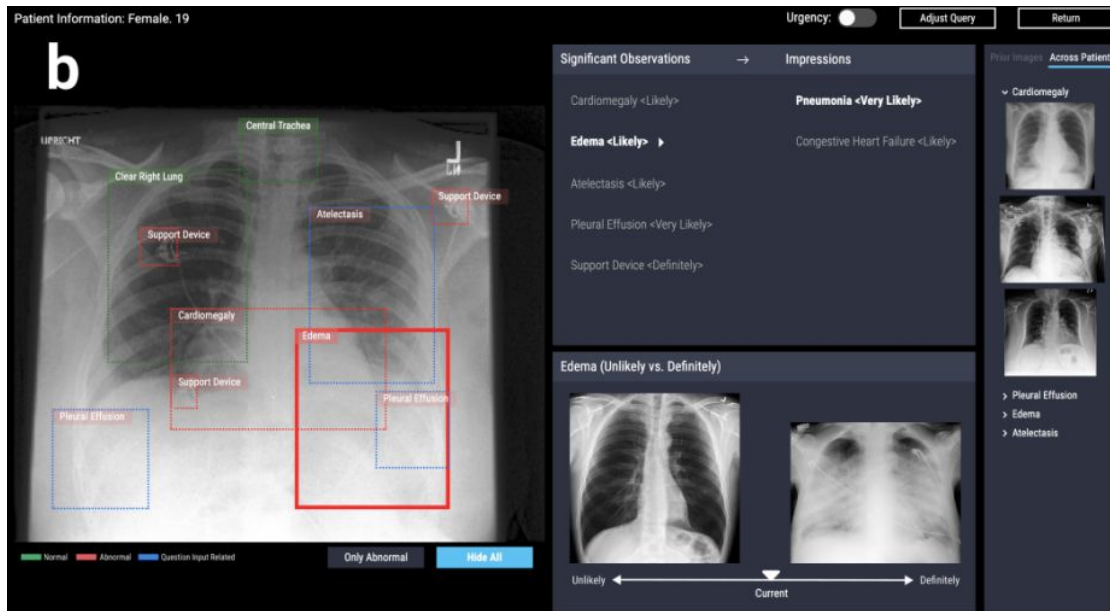


A giraffe standing in a forest with trees in the background.



Explainable Artificial Intelligence (XAI) - Exp. Locales

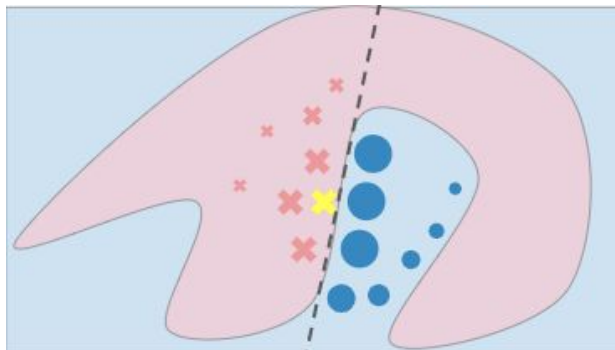
- Lo más utilizado son explicaciones locales, puesto que son más "sencillas" de entender.
- Es posible identificar 3 principales estrategias de explicación local:
 - **Analogías**



Explainable Artificial Intelligence (XAI)

Ejemplo de herramientas visuales - LIME

Ejemplos



Intuición

Posting	From: <code>livesey@solntze.wpd.sgi.com</code> (Jon <code>Livesey</code>)
0.07	Subject: <code>Re: A Little Too Satanic</code>
Host	Organization: <code>sgi</code>
0.07	Lines: 56
NNTP	Distribution: <code>world</code>
0.05	<code>NNTP-Posting-Host: solntze.wpd.sgi.com</code>
Re	
0.04	

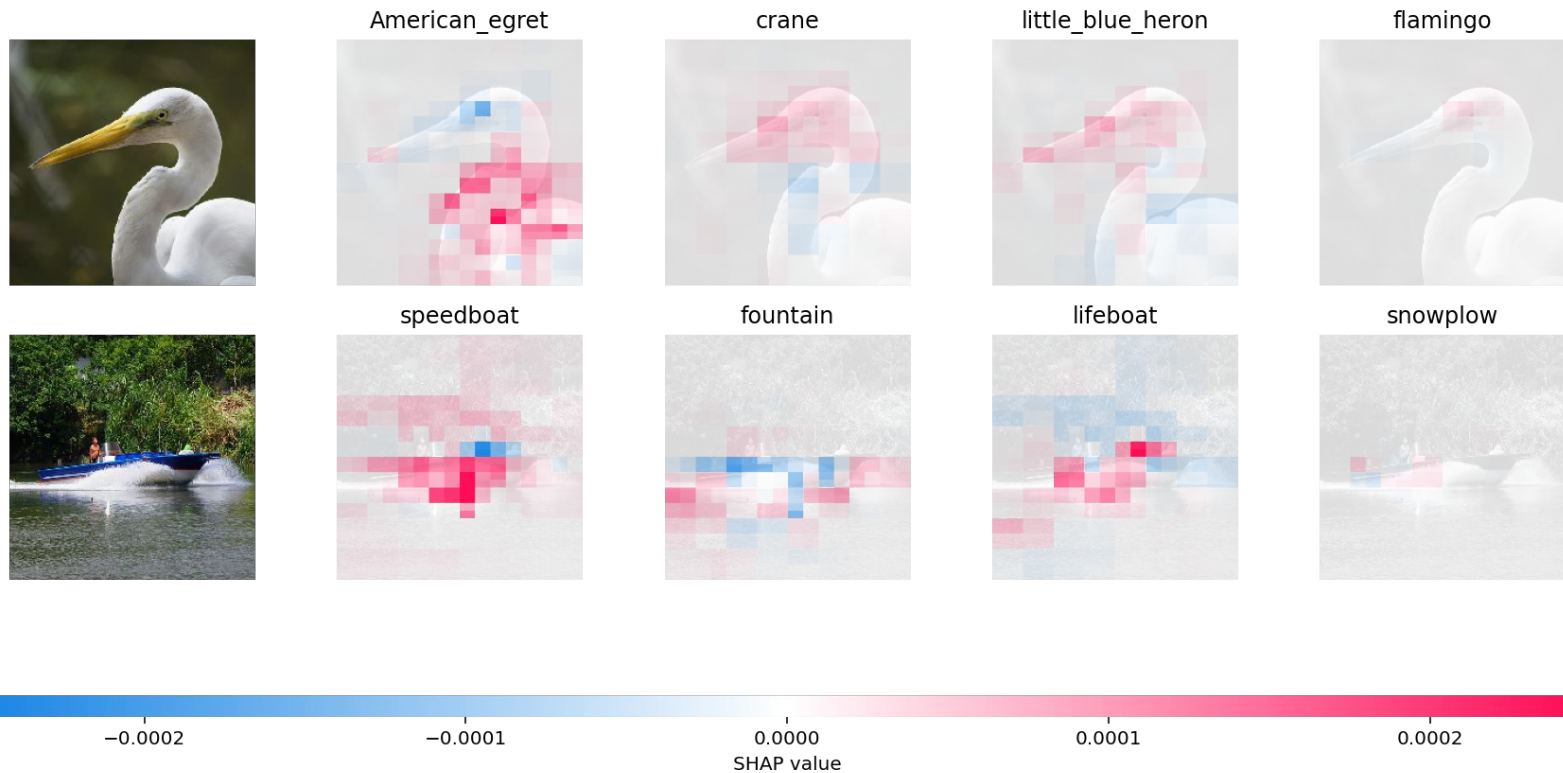
Probabilidad para ser clasificada cómo noticia económica: 85%

Diputados de oposición proponen bono para compensar histórica caída del **fondo E**

Conocido como el menos riesgoso, las y los pensionados se sorprendieron con el 10,4% de caída acumulada que presenta este **fondo**. Mientras parlamentarios de oposición **entregaron** una **propuesta** de **reparación**, **expertos** del **mercado** aseguran que el **origen** de la **pérdida** se encuentra en iniciativas aprobadas por el **propio** Congreso.

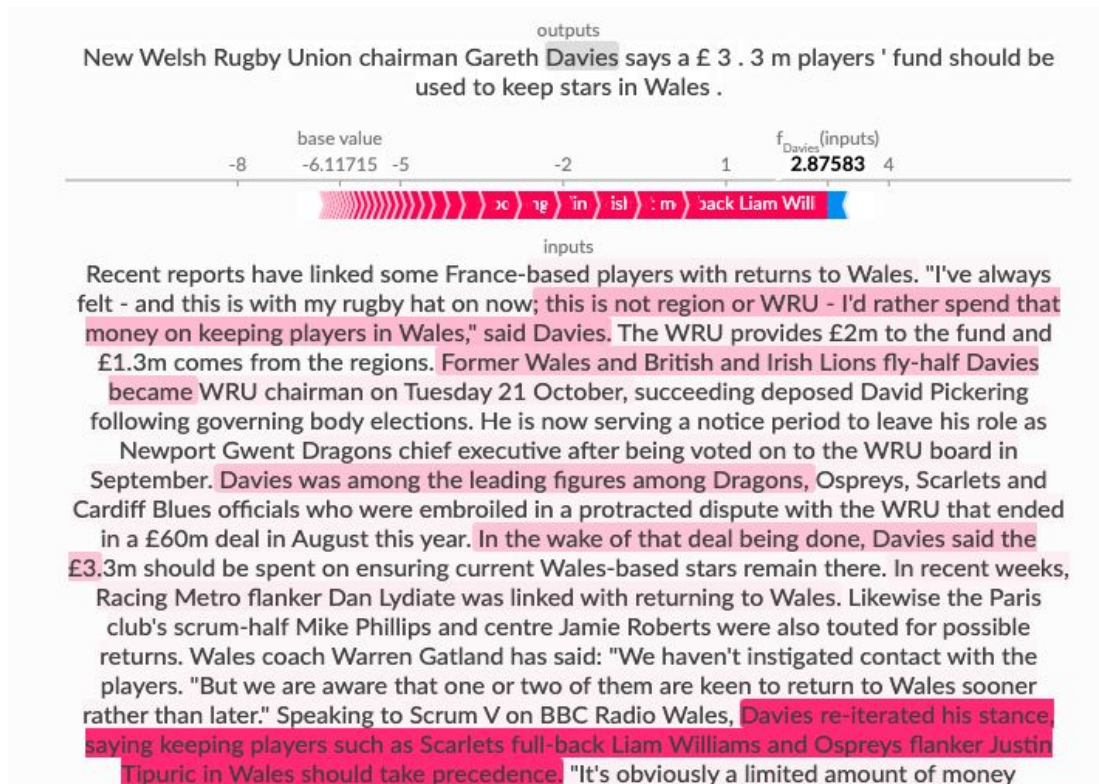
Explainable Artificial Intelligence (XAI)

Ejemplo de herramientas visuales - SHAP



Explainable Artificial Intelligence (XAI)

Ejemplo de herramientas visuales - SHAP



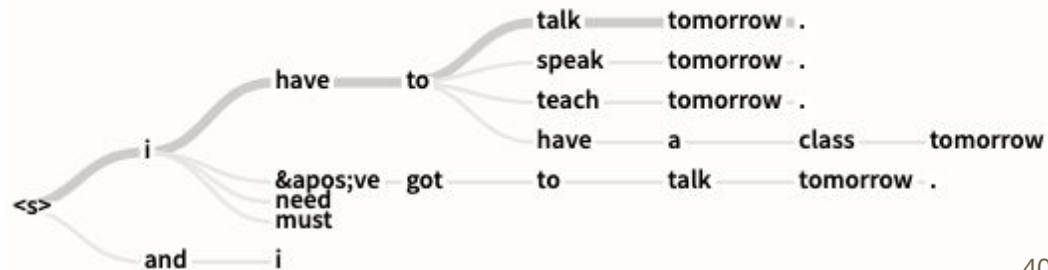
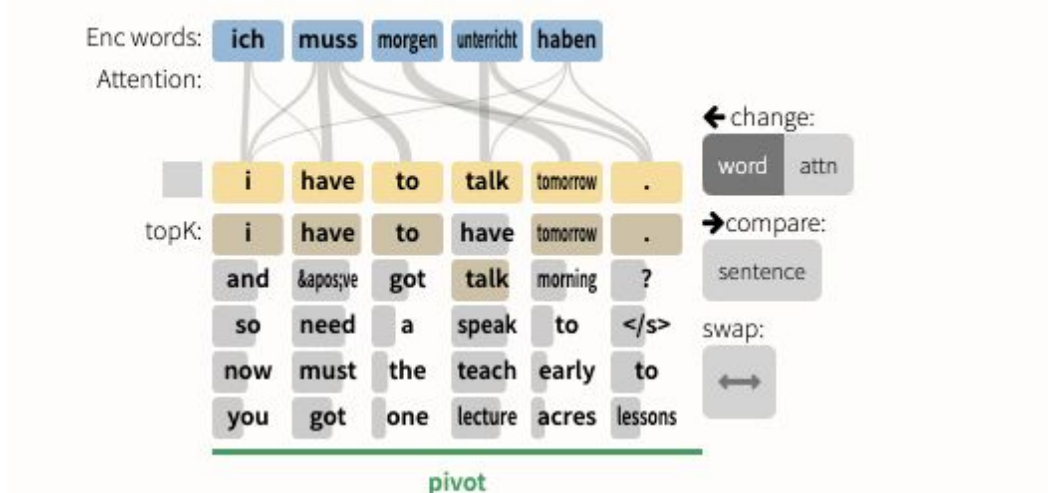
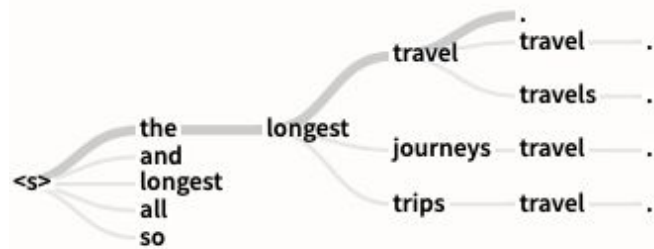
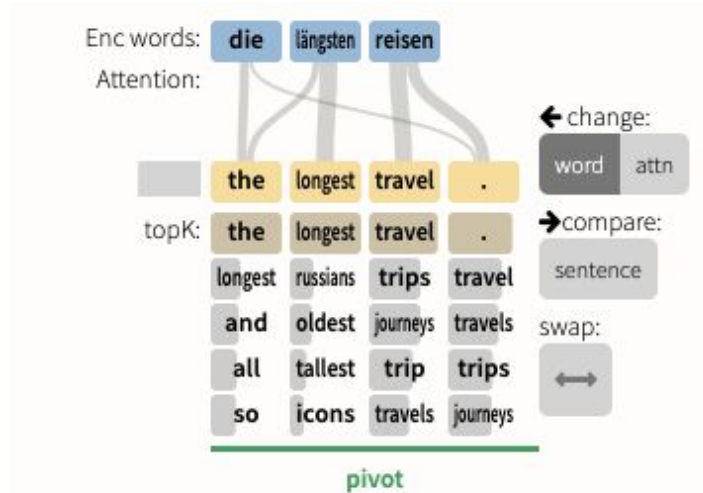
Explainable Artificial Intelligence (XAI)

Ejemplo de herramientas visuales - Mecanismo de atención en imágenes



Explainable Artificial Intelligence (XAI)

Ejemplo de herramientas visuales - Mecanismo de atención en traducción



Explainable Artificial Intelligence (XAI)

- 🤔 ¿Cómo diseño visualizaciones para XAI?
- 🤔 ¿Qué aspectos debo tener en cuenta?
- 🤔 ¿El diseñador entenderá que quiero visualizar el mecanismo de atención?

Tema interesante de investigación:

- [The State of the Art in Integrating Machine Learning into Visual Analytics](#)
- [Question-Driven Design Process for Explainable AI User Experiences](#)
- [Desarrollo y evaluación de un modelo de diseño de visualizaciones para inteligencia artificial explicable](#) (Mi tesis de magíster)

Explainable Artificial Intelligence (XAI)

Códigos de ejemplo con Python

- [Explain your model predictions with LIME | Kaggle](#)
- [Explain ResNet50 using the Partition explainer — SHAP](#)
- [Multi-input Gradient Explainer MNIST Example — SHAP](#)
- [Text to Text Explanation: Abstractive Summarization Example — SHAP](#)
- [Test RuleMatrix Render Example](#)

Streamlit

¿qué es y qué puedo hacer con él?

¿Qué es *Streamlit*?

- *Streamlit* es una librería open-source de Python, que nos permite crear aplicaciones web.
- Utilizada popularmente para *machine learning* y *data science*.
 - Puedes entrenar un modelo y dejarlo disponible para su uso.
- Su ventaja es que no es necesario aprender tecnologías web (JavaScript) para hacer visualizaciones interactivas.
- Tiene compatibilidad con Altair, Matplotlib, plotly, folium, entre otros.
- Documentación (100% recomendada): [Streamlit documentation](#)
- Excelente tutorial: [Curso Completo de Streamlit: Aprende a Crear Apps Web con Python desde Cero - YouTube](#)

¿Cómo funciona *Streamlit* por dentro?

```
import streamlit as st

st.title("Ejemplo")

edad = st.slider("Edad", 0, 100, 25)

st.write("Edad:", edad)

print("El script se ejecutó")
```

Ejemplo

Edad



Edad: 25

- *Streamlit* ejecuta todo el código y lo transforma en una aplicación web.
- Si un *widget* de *input* cambia (como `st.slider`), todo el código se ejecuta de nuevo para actualizar lo que corresponda.
- [Ejemplo](#)

¿Cómo funciona *Streamlit* por dentro?

Reduciendo ejecuciones costosas → **Interactividad propia del gráfico**

- En lugar de usar *widgets* externos para cada cambio, aprovechar que algunos gráficos permiten *zoom*, selección o filtros sin provocar nuevas ejecuciones del código.
 - Filtros incluidos en los gráficos (como altair).
 - Gráficos coordinados.

¿Cómo funciona *Streamlit* por dentro?

Reduciendo ejecuciones costosas → **Formularios (`st.form`)**

- Los *widgets* dentro del formulario no provocan una ejecución inmediata.
- Solo cuando el usuario presiona Enviar se ejecuta nuevamente el *script*.

```
with st.form("my_form"):
    st.write("Inside the form")
    slider_val = st.slider("Form slider")
    checkbox_val = st.checkbox("Form checkbox")

    submitted = st.form_submit_button("Submit")
    if submitted:
        st.write("slider", slider_val, "checkbox", checkbox_val)
```

¿Cómo funciona *Streamlit* por dentro?

Reduciendo ejecuciones costosas → **Caché** (`@st.cache_data`)

- Evita repetir cálculos o lecturas de archivos cuando los datos no han cambiado.
- La primera vez ejecuta la función y guarda el resultado; en las siguientes reutiliza el resultado almacenado mientras los parámetros no cambian.

```
@st.cache_data
def cargar_datos(file_path):
    return pd.read_csv(file_path)
```


¿Cómo funciona *Streamlit* por dentro?

Reduciendo ejecuciones costosas → **Uso de `session_state`**

- Guardar información entre ejecuciones.
- Cambiar el código a ejecutar según la información guardada.
- Ejemplo: [st.dialog - Streamlit Docs](#)

Funciones básicas - ¿Qué puedo hacer con *streamlit*?

- Mostrar textos con distintos formatos (títulos, Markdown, LaTeX, código, etc.).
- Visualizar tablas y datos.
- Incluir gráficos estáticos e interactivos.
- Agregar controles de interacción (botones, sliders, selectores, formularios, etc.).
- Organizar la interfaz mediante layouts (columnas, pestañas, contenedores, barra lateral).
- Mostrar imágenes, audio y video.
- Cargar y descargar archivos.
- Integrar bibliotecas de Python (Pandas, Matplotlib, Plotly, Altair, Scikit-learn, entre otras).

Funciones básicas: Texto

Elementos para incluir textos en diferentes formatos → [Text elements - Streamlit Docs](#)

- Título de nuestra app

```
st.title('This is a title')  
st.title('_Streamlit_ is :blue[cool] :sunglasses:')
```

This is a title

***Streamlit* is cool 🕶️**

Funciones básicas: Texto

Elementos para incluir textos en diferentes formatos → [Text elements - Streamlit Docs](#)

- Markdown

```
st.markdown("*Streamlit* is **really** ***cool***.")
```

```
st.markdown('''
:red[Streamlit] :orange[can] :green[write] :blue[text] :violet[in]
:gray[pretty] :rainbow[colors] and :blue-background[highlight] text.
''')
```

Streamlit is **really** *cool*.

Streamlit can write text in pretty colors and highlight text.

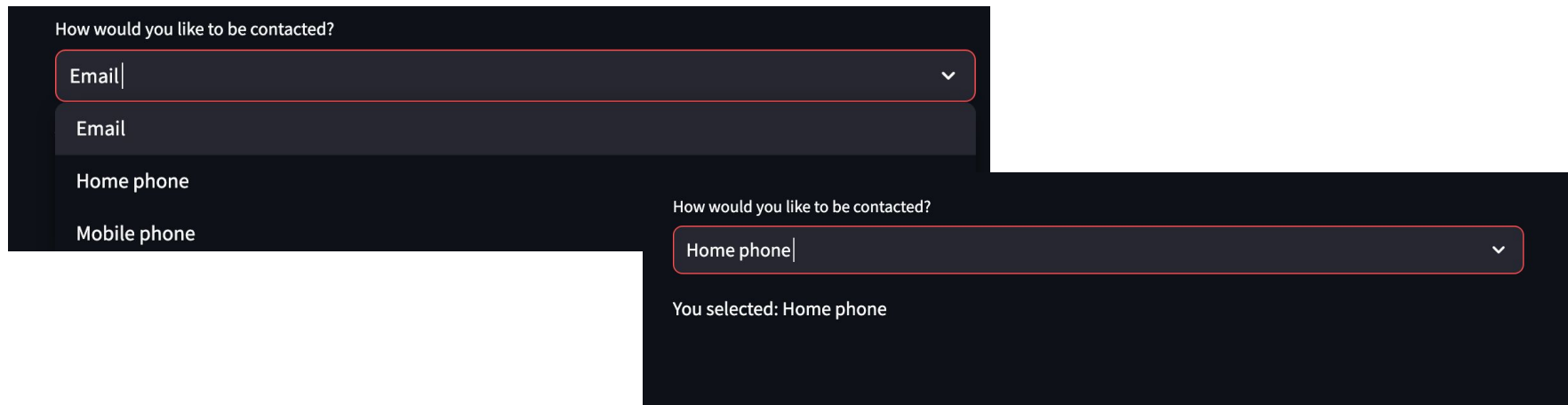
Funciones básicas: Input

Elementos para incluir interactividad → [Input widgets - Streamlit Docs](#)

- Selectores

```
option = st.selectbox(
    "How would you like to be contacted?",
    ["Email", "Home phone", "Mobile phone"])
```

```
st.write("You selected:", option)
```



How would you like to be contacted?

Email

Email

Home phone

Mobile phone

How would you like to be contacted?

Home phone

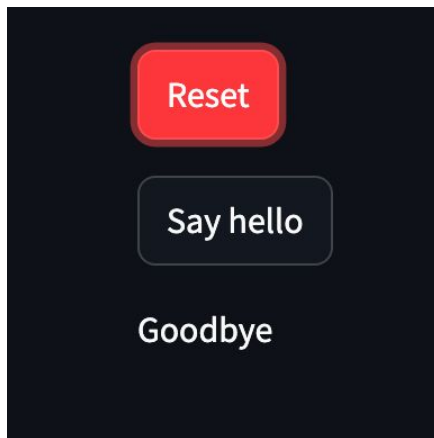
You selected: Home phone

Funciones básicas: Input

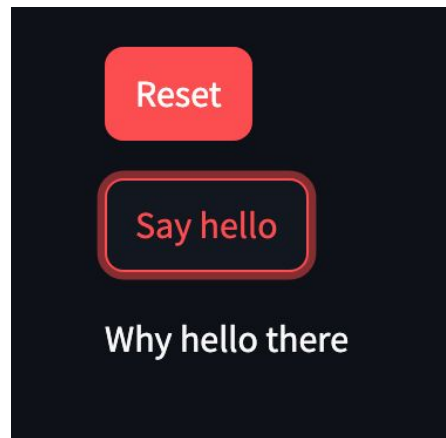
Elementos para incluir interactividad → [Input widgets - Streamlit Docs](#)

- Botón

```
st.button("Reset", type="primary")
if st.button("Say hello"):
    st.write("Why hello there")
else:
    st.write("Goodbye")
```



Presionar Reset



Presionar Say hello

Funciones básicas: Input

Elementos para incluir interactividad → [Input widgets - Streamlit Docs](#)

- *Slider*

```
age = st.slider("How old are you?", 0, 130, 25)  
st.write("I'm ", age, "years old")
```

Slider

How old are you?



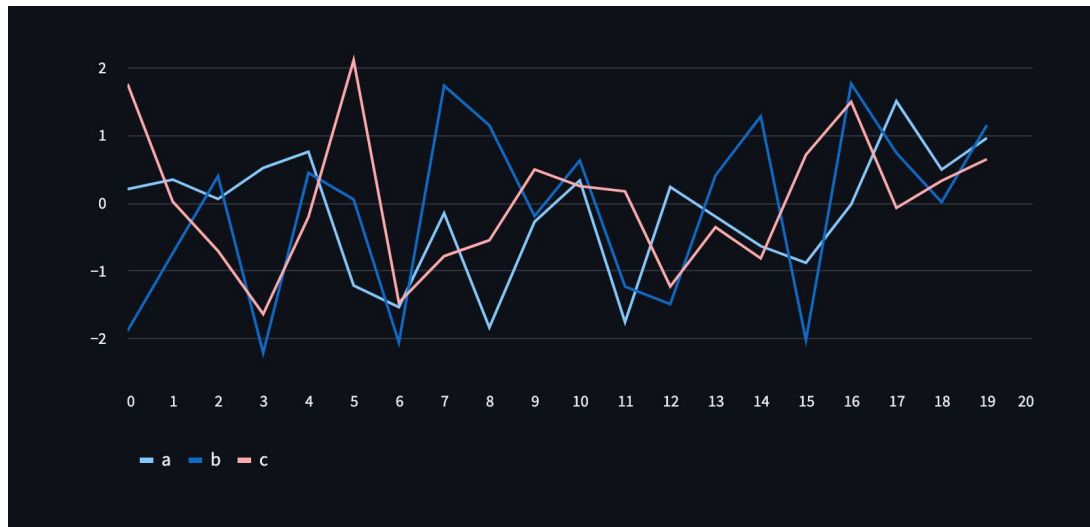
I'm 71 years old.

Funciones básicas: Gráficos

Elementos para agregar visualizaciones → [Chart elements - Streamlit Docs](#)

- Gráficos directos de *streamlit*

```
chart_data = pd.DataFrame(np.random.randn(20, 3), columns=["a", "b", "c"])
st.line_chart(chart_data)
```



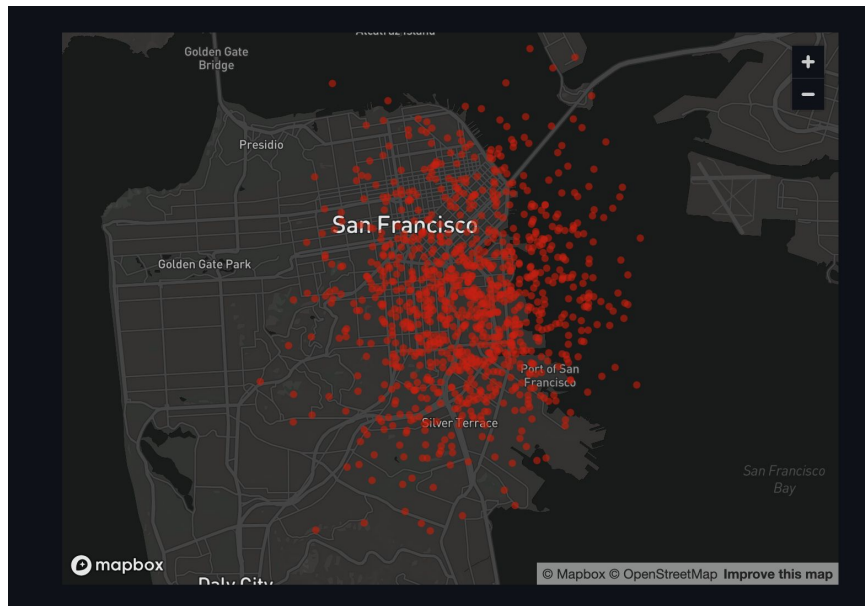
Funciones básicas: Gráficos

Elementos para agregar visualizaciones → [Chart elements - Streamlit Docs](#)

- Mapa de puntos

```
df = pd.DataFrame(np.random.randn(1000, 2) / [50, 50] + [37.76, -122.4],  
                  columns=['lat', 'lon'])
```

```
st.map(df)
```



Funciones básicas: Gráficos

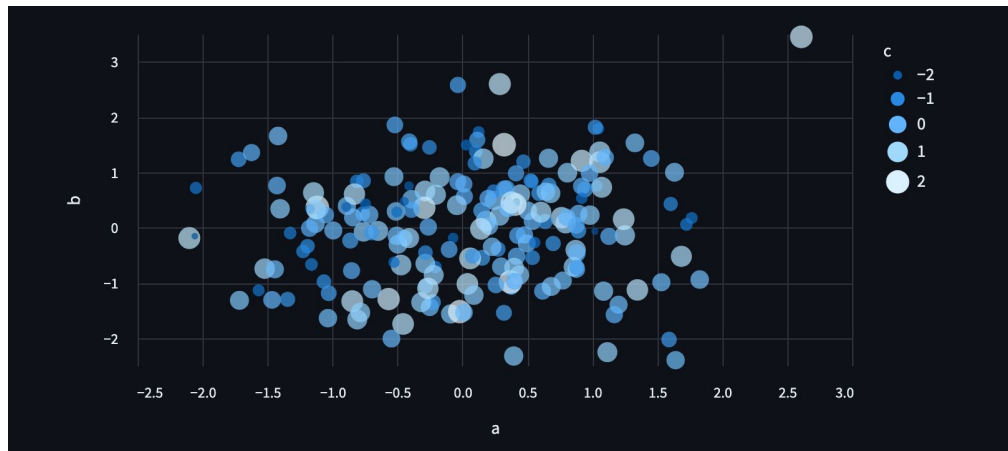
Elementos para agregar visualizaciones → [Chart elements - Streamlit Docs](#)

- Gráficos de Altair

```
chart_data = pd.DataFrame(np.random.randn(20, 3), columns=["a", "b", "c"])
```

```
c = alt.Chart(chart_data).mark_circle()  
    .encode(x="a", y="b", size="c", color="c", tooltip=["a", "b", "c"])
```

```
st.altair_chart(c)
```



Funciones básicas: Gráficos

Elementos para agregar visualizaciones → [Streamlit-Folium](#)

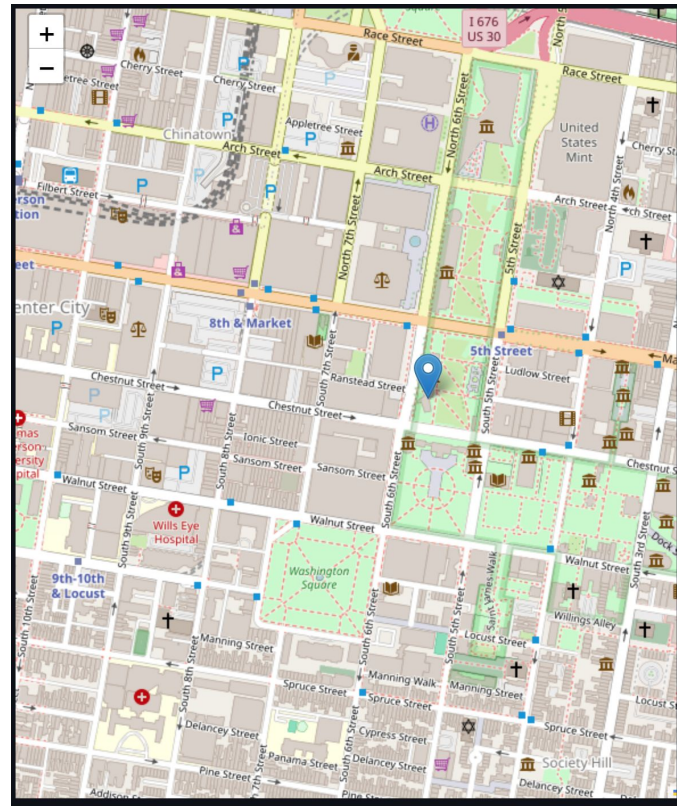
- Mapas de Folium

```
from streamlit_folium import st_folium
```

```
m = folium.Map(location=[39.949610, -75.150282],  
                zoom_start=16)
```

```
folium.Marker([39.949610, -75.150282],  
              popup="Liberty Bell",  
              tooltip="Liberty Bell").add_to(m)
```

```
st_data = st_folium(m, width=725)
```



Funciones básicas: Layout

Elementos para definir el layout → [Layouts and Containers - Streamlit Docs](#)

- Columnas

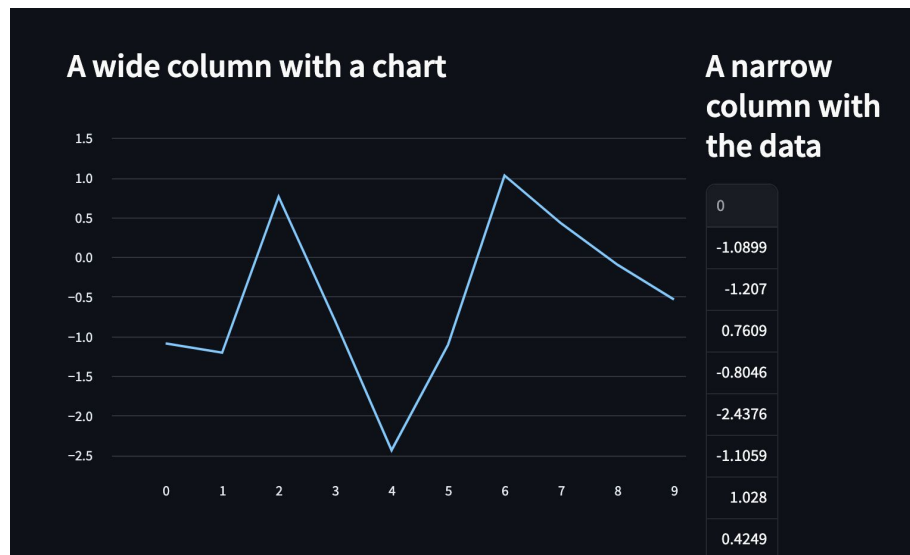
```
col1, col2 = st.columns([3, 1]) # Columna 1 es 3 veces más grande que columna 2
data = [-1.08, -1.20, 0.76, -0.80, -2.43, -1.10, 1.02, 0.42, -0.09, -0.53]
```

```
col1.subheader("A wide column  
with a chart")
```

```
col1.line_chart(data)
```

```
col2.subheader("A narrow  
column with  
the data")
```

```
col2.write(data)
```



Machine Learning y Streamlit

También podemos utilizar *Streamlit* como interfaz de nuestros modelos

1. Necesitamos incluir los *inputs* necesarios del modelo en el *dashboard*.
2. Tener un modelo entrenado y guardado
 - a. *joblib*, *pickle*, *keras* y *pytorch* tienen formas de lograr esto.
3. Procesar los *inputs* y solicitar al modelo el resultado.
4. Desplegar respuesta en la interfaz.

Ejemplos Aplicados

- [Sophisticated Palette](#)
- [Streamlit theme for Plotly charts!](#)
- [JULO improves financial inclusion in Indonesia with Streamlit](#)
- [Wissam Siblini uses Streamlit for pathology detection in chest radiographs](#)
- [How Delta Dental uses Streamlit to make lightning-fast decisions](#)

Streamlit

¿Cómo disponibilizo mi
herramienta?

¿Cómo disponibilizo mi *dashboard*?

1. Cuenta gratuita en [Streamlit Community Cloud](#)

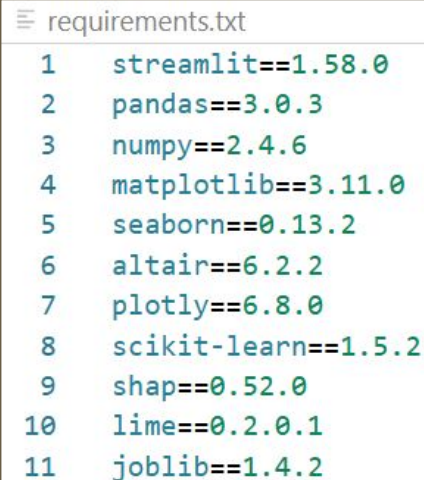
- ◆ Recomendación: Registrarse con la cuenta de Github.

2. Crear repositorio

- ◆ Archivo principal con streamlit (app.py)
- ◆ Incluir requirements.txt con librerías adicionales.
- ◆ El código debe funcionar para correr con el comando **streamlit run app.py**

3. Ir a <https://share.streamlit.io/new>

- ◆ Seguir los pasos con github



```
requirements.txt
1  streamlit==1.58.0
2  pandas==3.0.3
3  numpy==2.4.6
4  matplotlib==3.11.0
5  seaborn==0.13.2
6  altair==6.2.2
7  plotly==6.8.0
8  scikit-learn==1.5.2
9  shap==0.52.0
10 lime==0.2.0.1
11 joblib==1.4.2
```


¿Cómo disponibilizo mi *dashboard*?

[← Back](#)

Deploy an app

Repository [?](#)

[Paste GitHub URL](#)

hernan4444/repo

Branch

master

Main file path

streamlit_app.py

App URL (optional)

.streamlit.app

[Advanced settings...](#)

Deploy!

Streamlit

Presentación de una herramienta
ya publicada

Herramienta en producción

Contexto

- Registros de Airbnb en 10 países distintos
 - ID del anfitrión
 - Ubicación (latitud, longitud)
 - tiempo de respuesta
 - Puntaje según localización y comunicación
 - Capacidad y cantidad de camas
 - Si tiene o no ciertos servicios (wifi, aire acondicionado, etc)
- Vamos a explorar el *dataset* e intentaremos predecir en qué país corresponde según sus datos. Agregaremos explicación a tal clasificación.

Herramienta en producción

Vistas

- Inicio
 - Entregar información básica del *dataset*.
- Visualizaciones
 - Explorar el *dataset* con visualizaciones de altair, seaborn y plotly.
- XAI
 - Predecir el continente del *airbnb* en función de sus atributos.
 - Explicar cuáles fueron los atributos más relevantes para dicha clasificación.

Herramienta en producción

Enlace a la demo

Herramienta en producción

Enlace al código



Streamlit

¿Existen alternativas?

Alternativas - *Gradio*

Users > abidlabs > Desktop > app.py > ...

```
1  from model import generate_image
2
3  import gradio as gr
4
5  with gr.Blocks() as demo:
6      gr.Markdown("# Your Fancy Image Generator")
7      with gr.Row(equal_height=True):
8          textbox = gr.Textbox(lines=1, show_label=False, placeholder="Enter prompt")
9          button = gr.Button("Generate", variant="primary")
10         image = gr.Image(height=250)
11
12         button.click(
13             generate_image,
14             inputs=textbox,
15             outputs=image,
16         )
17
18     demo.launch(share=True)
19     # Creates a public link like:
20     # https://b25f924d951331d187.gradio.live
```

Your Fancy Image Generator

cheetah

Generate

Image



Alternativas - *Gradio*

Ventajas

- Muy fácil de aprender y utilizar.
- Ideal para demostrar modelos de IA y *Machine Learning*.
- Permite compartir aplicaciones rápidamente.
- Incluye componentes preparados para texto, imágenes, audio y video.

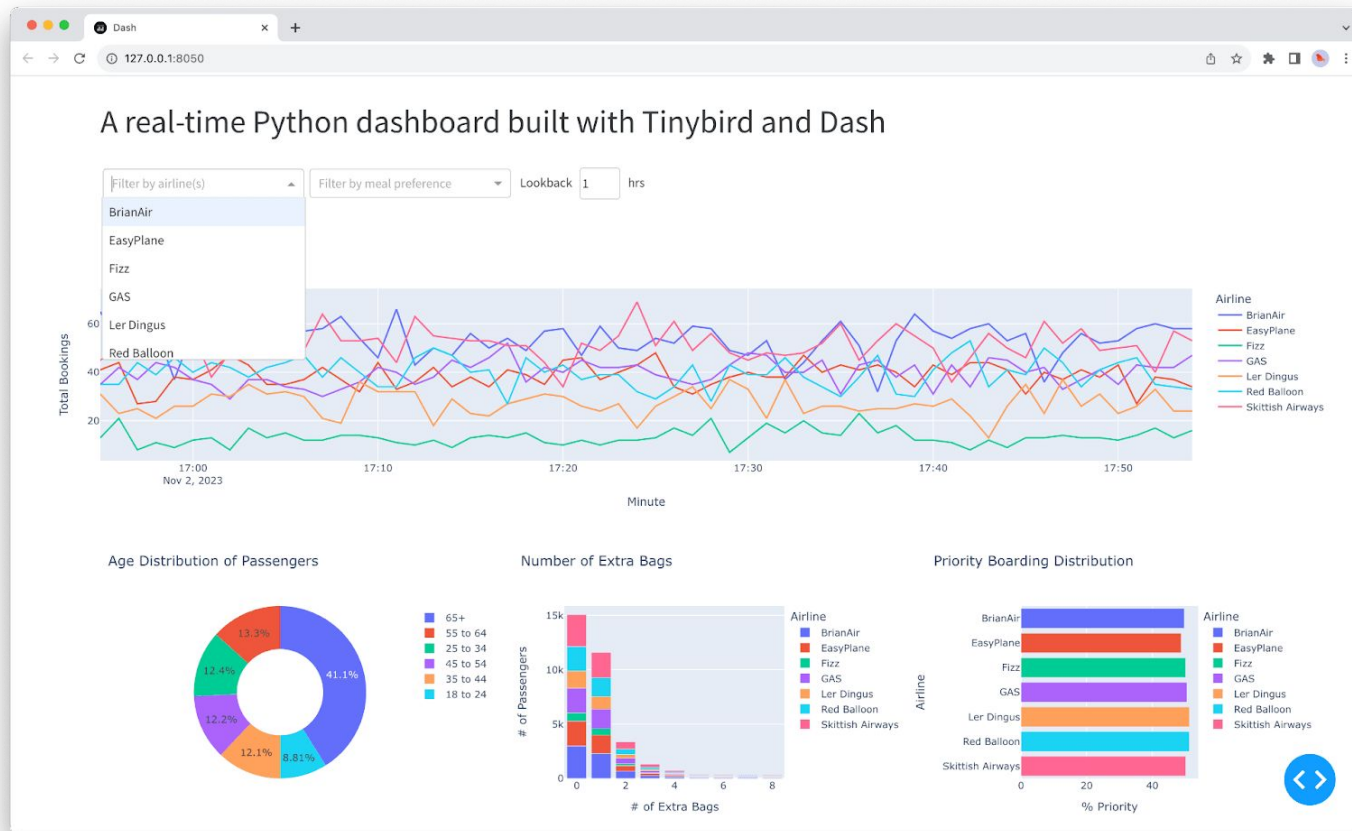
Desventajas

- Menos flexible para *dashboards* complejos.
- Personalización de la interfaz más limitada.
- Pensado principalmente para aplicaciones de IA.

Recursos

- [Documentación](#)
- [Ejemplos en Spaces](#)

Alternativas - *Dash*



Alternativas - *Dash*

Ventajas

- Muy flexible para *dashboards* complejos.
- Excelente integración con Plotly.
- Amplia personalización de la interfaz.
- Muy utilizado en aplicaciones empresariales.

Recursos

- [Documentación](#)
- [Ejemplos](#)

Desventajas

- Curva de aprendizaje mayor más alta. Requiere comprender la estructura de una página web (componentes similares a HTML).
- La interacción se implementa mediante callbacks, donde es necesario declarar qué componentes reaccionan a cuáles eventos.
- El despliegue suele requerir un servidor o servicio externo.

Alternativas

Característica	<i>Streamlit</i>	<i>Dash</i>	<i>Gradio</i>
Facilidad de aprendizaje	★★★★★	★★	★★★★★
Flexibilidad	★★	★★★★★	★
<i>Dashboards</i>	★★★★	★★★★★	★
IA / <i>Machine Learning</i>	★★★★	★★	★★★★★
Rapidez para prototipos	★★★★★	★★	★★★★★

- ***Streamlit***: equilibrio entre simplicidad y funcionalidad.
- ***Dash***: para *dashboards* altamente personalizados.
- ***Gradio***: para demos de modelos de IA.



Visualización de Información y Analítica Visual

— Hernán Valdivieso López (hfvdivieso@uc.cl) —
